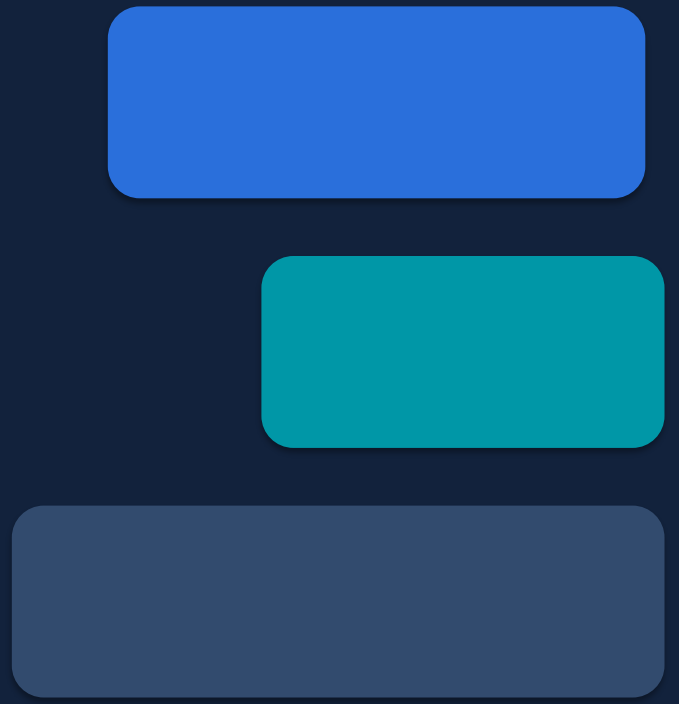
AGENTS & AGENTIC AI



Concepts • Architecture • Tool execution • Workflows • Governance

A practical learning deck with Spring AI and customer-support examples

2026

Learning map

From a basic LLM call to an autonomous, governed system

1. Foundations

LLM, prompt, context, tools and memory

2. Agent

Goal, reasoning, action and observation

3. Agentic system

Planning, loops, state and adaptation

4. Production

Safety, evaluation, monitoring and control

- Understand the difference between an agent and agentic behaviour.
- Trace how Spring AI coordinates an LLM and tool callbacks.
- Design a customer-support workflow that mixes deterministic code and LLM decisions.

Start with the building blocks

An agent is built on top of ordinary application and LLM capabilities

LLM

Generates or interprets language from the supplied context. It predicts the next output; it does not automatically know your private business data.

Prompt + context

System instructions, user request, conversation history, retrieved documents and tool descriptions shape the model's response.

Tool

A named capability with an input schema and executable implementation: Java method, API, database operation or MCP tool.

Application

Owns authentication, business rules, data access, validation, transactions, retries and observability.

Key idea: the LLM proposes; application code and tools perform controlled operations.

What is an AI agent?

A useful engineering definition

An AI agent is a software system that uses an AI model to interpret a goal, decide what action to take, use available tools, observe results and continue until it reaches a stopping condition.

Goal

What outcome should be achieved?

Decision

Which step or tool is appropriate next?

Action

Call a tool or produce an answer.

Observation

Inspect result and decide whether to continue.

An agent is more than a prompt and more than a single tool call.

Agent vs. agentic AI

Related terms, but they describe different levels

AI agent

A component or runtime that pursues a goal through a decision–action–observation cycle.

Example: a refund-support agent that checks transactions and prepares a refund request.

Agentic system / agentic AI

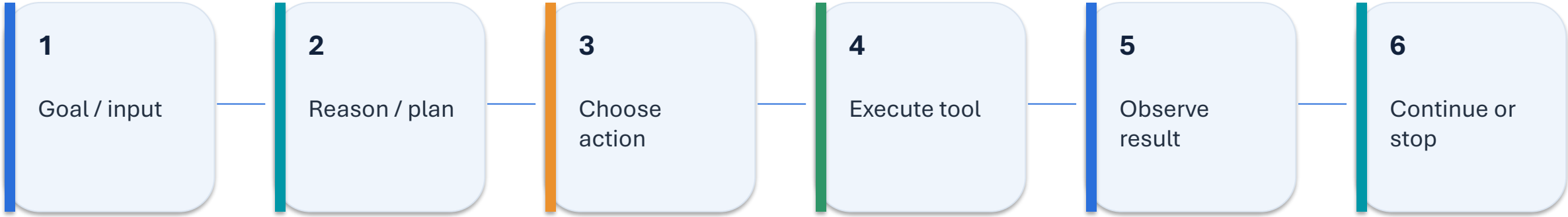
A system exhibiting goal-directed, multi-step behaviour: planning, tool use, state tracking, adaptation and sometimes coordination of multiple agents.

Example: a support platform coordinating ticket, billing, knowledge and notification capabilities.

An agent is often one building block; agentic describes the broader behaviour or system design.

The agent execution loop

The recurring cycle behind tool-using agents



The loop may repeat several times. The model can request a tool, receive its result, and then request another tool—or finish with a natural-language response.

Stop conditions: goal achieved • no useful next action • max iterations • timeout • human approval required

Core components of an agent

Separate reasoning from execution and governance

Model

Interprets context and selects next step.

Instructions

Role, objective, constraints and policies.

Tools

APIs, Java methods, MCP servers, databases.

Memory / state

Conversation history, task state and durable facts.

Planner / orchestrator

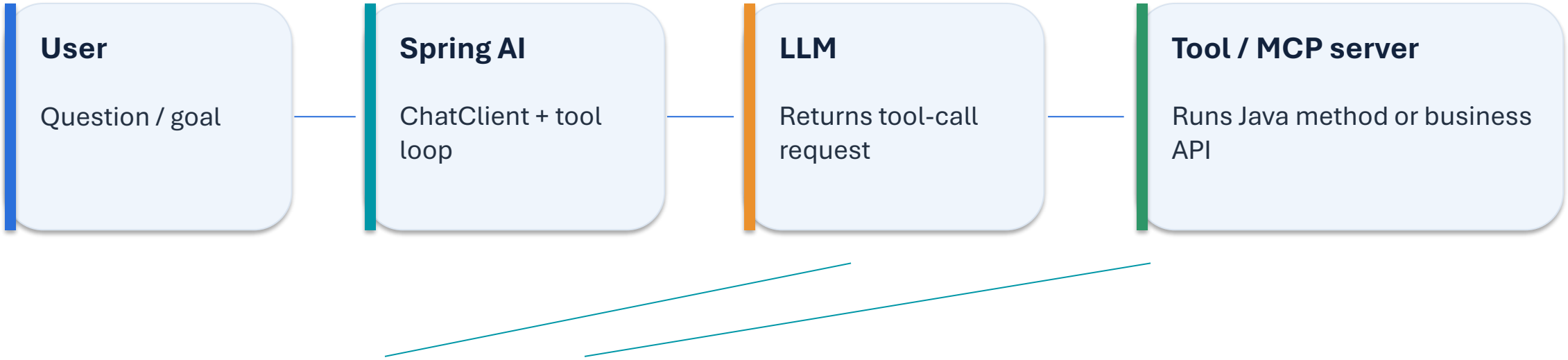
Controls sequence, retries, branching and hand-offs.

Guardrails

Authorization, validation, approvals and limits.

LLM + tools: who does what?

The model does not directly connect to your MCP server



Spring AI sends tool definitions to the model, receives the requested tool name and arguments, executes the matching callback, and returns the result to the model as conversation context.

LLM = decision about what to call | Spring AI = tool-call orchestration | Tool = actual operation

One-by-one or multiple tools in one round?

The dependency between actions determines the useful sequence

Dependent tools — sequential

1. getBillingTransactions
   ↓ result
2. checkDuplicatePayment
   ↓ result
3. createRefundRequest
   ↓ result
4. LLM creates final explanation

Independent tools — may be batched

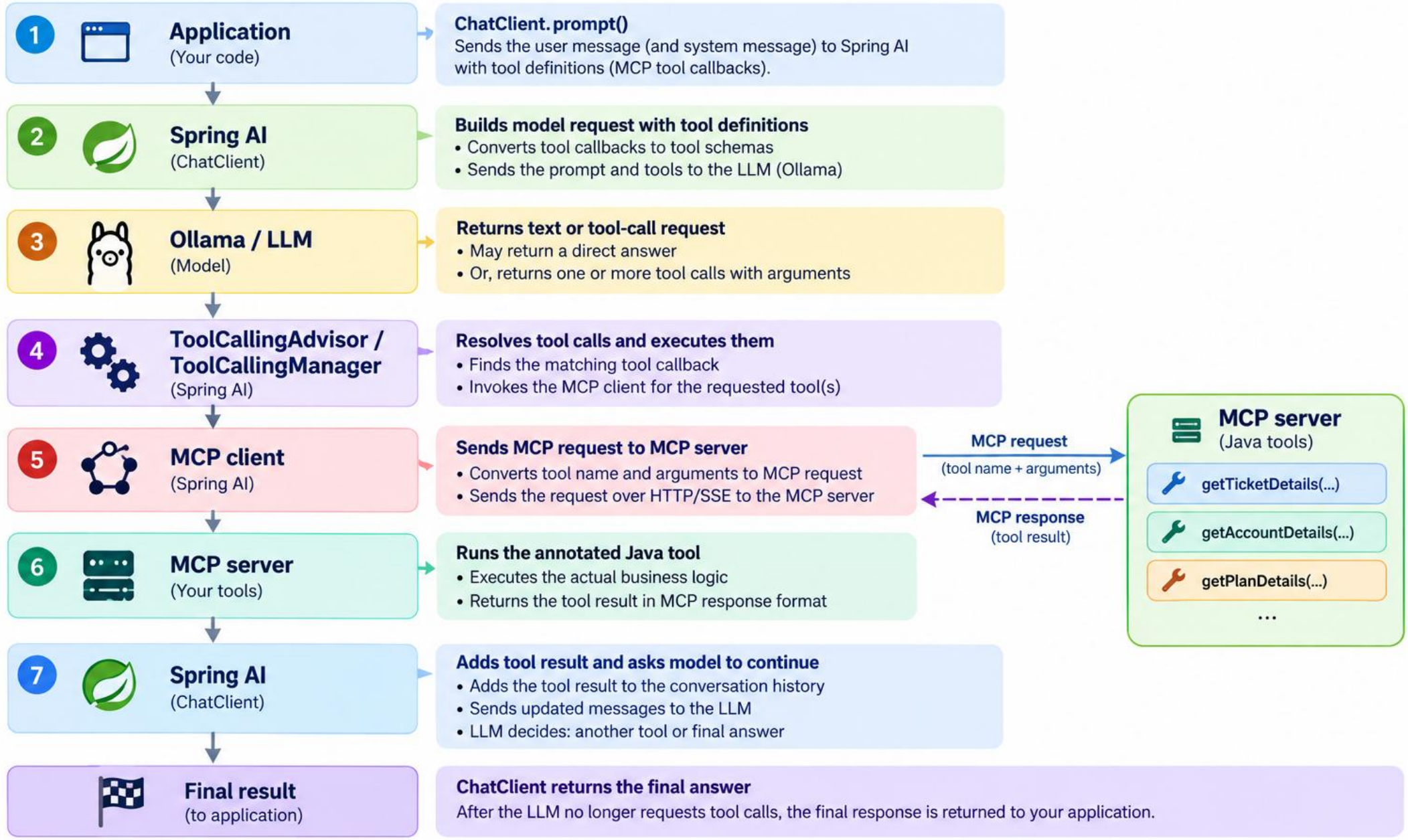
Example: fetch ticket details + retrieve customer plan + look up policy.

If the model requests these together, Spring AI can execute multiple tool calls in the same round and return their results for the next model turn.

Important: exposing three tools does not mean all three will be called. The model chooses based on the request, instructions and tool descriptions.

Spring AI: the tool-calling path

Conceptual flow for ChatClient with MCP tool callbacks



MCP: a standard way to expose capabilities

Model Context Protocol separates capability providers from AI clients

MCP server

Publishes capabilities:

- Tools — executable actions
- Resources — readable context/data
- Prompts — reusable prompt templates

MCP client

Connects to server, discovers capabilities and adapts them for the host application.

In Spring AI, client configuration and callbacks/resources/prompts are separate concerns.

AI application

Chooses when to use tools, which resources to read, and how prompt content is combined with the user request.

Tools are not the same as resources or prompts. `.tools(...)` exposes tool callbacks; resources and prompts generally need explicit client calls and application handling.

Customer-support example: duplicate payment

A goal requiring several dependent actions

CUSTOMER REQUEST

“Please check customer C1001. If a duplicate payment exists, start a refund request and explain the next steps.”

1. **Understand**
   Identify customer and subscription/payment scope.
2. **Retrieve**
   Call getBillingTransactions to fetch payment records.
3. **Verify**
   Call checkDuplicatePayment using the retrieved transactions.
4. **Act safely**
   Only if a duplicate is confirmed, call createRefundRequest (subject to policy and approval).
5. **Communicate**
   Explain the refund-request status and next steps; notify the customer when appropriate.

Tool call sequence

Tools invoked by the agent (as needed)

getBillingTransactions
(Fetch payment records)

→

checkDuplicatePayment
(Verify duplicate payment)

→

createRefundRequest
(Create refund if duplicate is confirmed)

Business rule: Never create a refund request unless duplicate payment is verified.

Common agentic patterns

Choose the simplest pattern that solves the problem

Single-agent loop

One model repeatedly selects tools and evaluates results.

Router

Classifies the request and sends it to a specialist workflow.

Planner–executor

Planner breaks a goal into steps; executor performs them.

Multi-agent collaboration

Specialist agents exchange tasks or results under a coordinator.

Human-in-the-loop

Agent prepares a consequential action; person approves it.

Event-driven agent

A trigger starts a bounded task, such as investigating a failed payment.

Agent or deterministic workflow?

Use autonomy where it adds value; use code for invariants

Deterministic workflow

Best when:

- Steps and order are known
- Financial or compliance rules must always hold
- Repeatability and auditability matter
- Transaction boundaries are important

Agentic approach

Best when:

- User intent is ambiguous
- Next step depends on observations
- Tool choice varies by case
- The task needs flexible investigation or synthesis

Production recommendation: hybrid design — Java enforces business rules and permissions; the LLM handles language, interpretation and bounded decisions.

Safety, governance and reliability

Autonomy must be bounded

1 **Identity & authorization**	• Check who is asking and whether they may access the customer or action. • Enforce authentication, authorization and role-based access.
2 **Input validation**	• Validate tool arguments, types, ranges and required fields. • Reject invalid, unsafe or malicious input.
3 **Least privilege**	• Expose only the tools and data needed for the task. • Restrict access based on user role, tenant and data scope.
4 **Approval gates**	• Require human approval for sensitive actions such as refunds, account changes or external messages. • Use policy-based approvals and business rules.
5 **Limits**	• Set max tool calls, timeouts, budgets, retries and rate limits. • Prevent infinite loops and control resource usage.
6 **Audit & observability**	• Record user goal, model decision, tool name, arguments, result, approvals and outcome. • Monitor for misuse, errors and unexpected behavior.

Memory, RAG and state are different

Do not treat every piece of context as agent memory

Conversation history

Recent messages and tool results supplied to the model so it can follow the current conversation.

RAG knowledge

External documents retrieved by semantic or hybrid search and inserted as grounded context.

Task state / memory

Structured progress, decisions, preferences or durable facts stored under explicit policies.

Example task state: ticketId=T1001; duplicateCheck=CONFIRMED; refundRequest=CREATED; customerNotified=false.

How to test an agentic system

Evaluate the whole trajectory, not only the final answer

- Task success: did it reach the requested outcome?
- Tool correctness: did it choose the right tool and provide valid arguments?
- Sequence correctness: did it respect dependencies and policy conditions?
- Grounding: were answers supported by tool results or retrieved documents?
- Safety: did it avoid unauthorized or unapproved actions?
- Efficiency: how many model turns, tool calls, tokens, seconds and dollars?
- Resilience: what happens on tool timeout, malformed result, duplicate request or partial failure?
- Traceability: can an engineer reconstruct every decision and side effect?

Spring AI: a compact tool-enabled call

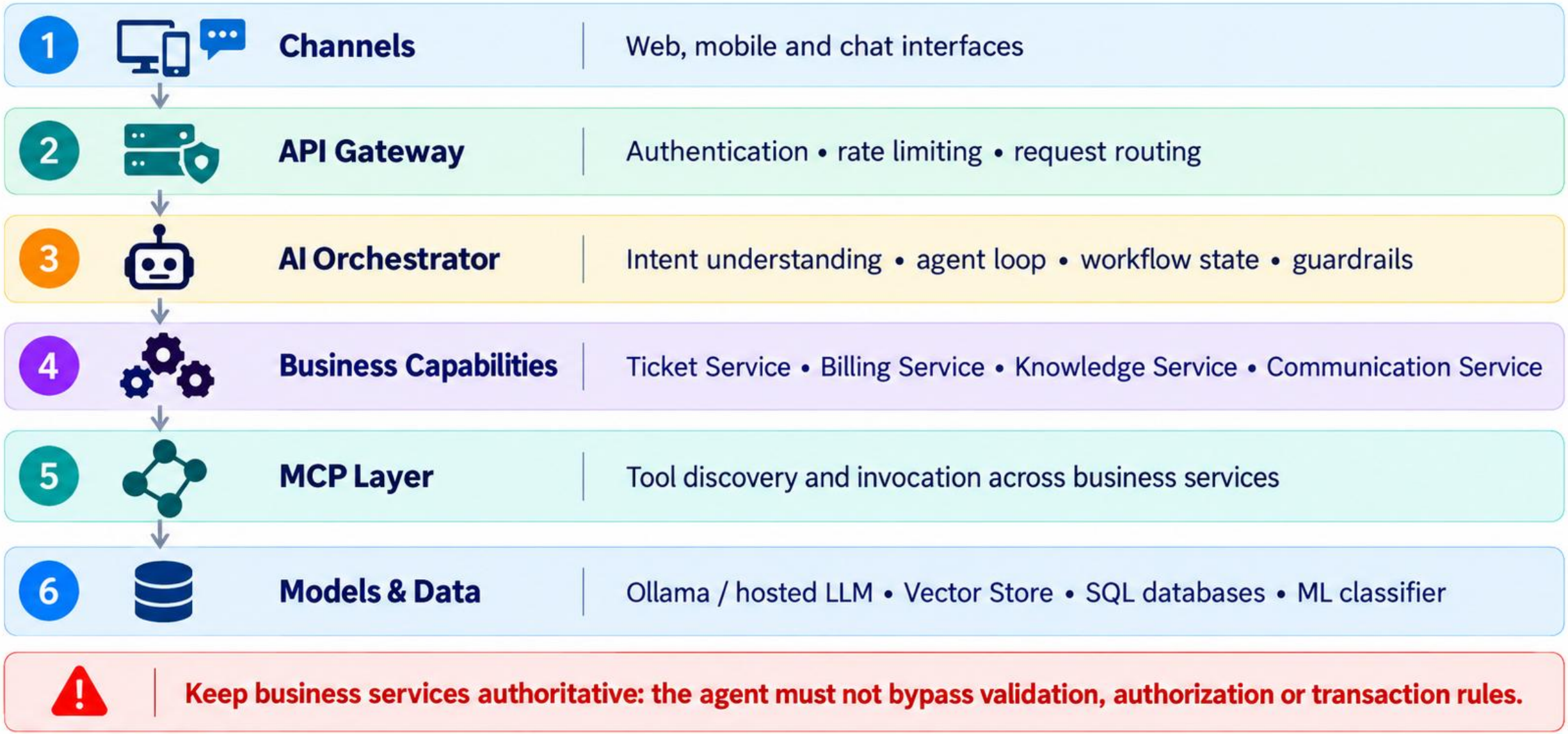
Illustrative Java pattern — exact APIs depend on your Spring AI version

```
String answer = chatClient.prompt()
    .system("""
        You are a customer-support assistant.
        Retrieve transactions before checking for duplicates.
        Create a refund request only after a duplicate is confirmed.
        Never claim a refund is completed unless the tool confirms it.
        """)
    .user("""
        Check customer C1001 for duplicate payments.
```

The model decides whether and when to call an exposed tool. Spring AI handles callback execution and returns tool results to the model. For critical ordering and financial invariants, enforce the rules in Java as well.

Reference architecture: agentic customer support

A hybrid architecture for the example project



Key takeaways

- ✓ An agent uses a model to pursue a goal through decisions, actions and observations.
- ✓ Agentic AI describes broader goal-directed, adaptive, multi-step behaviour.
- ✓ Tools give the model controlled access to real capabilities; Spring AI executes callbacks.
- ✓ MCP standardizes how tools, resources and prompts are exposed to AI applications.
- ✓ Independent tool calls may be batched; dependent actions need results between steps.
- ✓ Use a hybrid architecture: model flexibility + deterministic business controls + human approval.

Design for useful autonomy—not maximum autonomy.

Glossary

Terms used throughout the presentation

Agent

Goal-directed system that observes, decides and acts.

Agentic AI

AI behaviour/system that can pursue multi-step goals with some autonomy.

Tool calling

Model requests an application to execute a named capability.

Orchestration

Controls the sequence and coordination of steps.

MCP

Model Context Protocol; standard for exposing tools, resources and prompts.

RAG

Retrieval-Augmented Generation; retrieves external knowledge for model context.

Human-in-the-loop

Person reviews or approves a decision/action.

Guardrail

Constraint that limits unsafe or unauthorized behaviour.

Further study and implementation checklist

- Start with one tool and a bounded use case; add tools only when needed.
- Write clear tool names, descriptions, schemas and error responses.
- Keep authorization and business invariants in application/service code.
- Test successful, negative, ambiguous, timeout and partial-failure paths.
- Inspect traces: model request, tool-call request, callback, result and final response.
- Check the official Spring AI reference for the exact version in your build; framework APIs evolve.
- Suggested next exercise: build a refund-support flow with a read-only transaction lookup, duplicate verification and an approval-gated refund request.

Your two implementations: what is different?

Both use the same manual Spring AI tool-calling loop; their goals and instructions differ.

Version 1 — investigate() | [AGENTIC]

1. Explicit goal: investigate before answering
2. Detailed investigation rules
3. Ticket lookup is required for ticket investigations
4. Account/plan lookup is required for SLA questions
5. Reassesses missing information after each result
6. Maximum 8 tool-execution cycles

Version 2 — ask() | [AGENT]

1. General goal: answer the customer question
2. Shorter, less prescriptive system message
3. Uses tools when account/ticket/plan facts are needed
4. Does not specify a detailed investigation sequence
5. Reviews tool results and answers
6. Maximum 5 tool-execution cycles

Important: [AGENT] and [AGENTIC] are log labels. Technically, both implementations are LLM-driven tool-calling loops with Java-controlled execution.

Code walkthrough 1 — AgenticSupportService

Your first version: goal, tools, model options and investigation instructions.

```
@Service
public class AgenticSupportService {
    private static final Logger log =
        LoggerFactory.getLogger(AgenticSupportService.class);

    private static final int MAX_STEPS = 8;

    private final ChatModel chatModel;
    private final ToolCallingManager toolCallingManager;
    private final ToolCallback[] toolCallbacks;

    public AgenticSupportService(
            ChatModel chatModel,
            ToolCallingManager toolCallingManager,
            SupportTools supportTools,
            AccountLookupTool accountLookupTool) {
        this.chatModel = chatModel;
        this.toolCallingManager = toolCallingManager;
        this.toolCallbacks =
            ToolCallbacks.from(supportTools, accountLookupTool);
    }
```

Explanation: ToolCallbacks.from(...) makes these Java tool methods available to the model; it does not execute them yet.

Code walkthrough 1 — the goal is defined in the SystemMessage

The SystemMessage tells the model what outcome to pursue and how to investigate.

```
Prompt prompt = new Prompt(List.of(
    new SystemMessage("""
        You are Aria, an agentic customer-support assistant.

        Your goal is to investigate the customer's question using tools
        before providing the final answer.

        Work autonomously within this single request, Investigation rules:
        1. Identify every piece of information needed to answer the question.
        2. For a ticket investigation, call the ticket lookup tool.
        3. If the question asks about a support plan, SLA, or response target,
           call the account/plan lookup tool as well.
        4. Do not provide the final answer until all required information
           has been retrieved, unless a required tool fails.
        5. After each tool result, reassess what information is still missing.
        6. Request additional tools when necessary.
        7. Use only actual tool results. Never invent ticket, account,
           plan, or SLA details.
        8. When the investigation is complete, provide a concise answer
           supported by the retrieved data.
        """),
    new UserMessage("""
        CUSTOMER GOAL:
        %
```

Teaching point: SystemMessage defines the goal and rules; UserMessage supplies the particular customer goal.

Code walkthrough 1 — the agentic decision/action loop

The model observes each tool result and chooses whether another action is needed.

```
for (int step = 1; step <= MAX_STEPS; step++) {
    if (!response.hasToolCalls()) {
        String answer = response.getResult().getOutput().getText();
        return answer;                    // Model has finished
    }

    var calls = response.getResult().getOutput().getToolCalls();
    log.info("LLM selected actions: {}",
        calls.stream().map(call -> call.name()).toList());

    ToolExecutionResult execution =
        toolCallingManager.executeToolCalls(prompt, response);

    if (execution.returnDirect()) {
        var history = execution.conversationHistory();
        return history.get(history.size() - 1).getText();
    }

    prompt = new Prompt(execution.conversationHistory(), options);

    if (step == MAX_STEPS) {
```

ToolCallingManager executes the requested callbacks; the next chatModel.call(...) gives the model the updated conversation history.

Code walkthrough 2 — general Agent implementation

Your second version: same mechanics, but a simpler objective and shorter instructions.

```
private static final int MAX_ITERATIONS = 5;

public AgenticSupportService(
        ChatModel chatModel,
        ToolCallingManager toolCallingManager,
        SupportTools supportTools,
        AccountLookupTool accountLookupTool) {
    this.chatModel = chatModel;
    this.toolCallingManager = toolCallingManager;
    this.toolCallbacks =
        ToolCallbacks.from(supportTools, accountLookupTool);
}

public String ask(String question) {
    OllamaChatOptions options = OllamaChatOptions.builder()
        .model("llama3.2:latest")
        .toolCallbacks(toolCallbacks)
        .temperature(0.1)
        .build();

    Prompt prompt = new Prompt(List.of(
        new SystemMessage("""
```

Code walkthrough 2 — manual execution loop

The second implementation executes the same decision → tool → observation cycle.

```
for (int iteration = 1;
     iteration <= MAX_ITERATIONS; iteration++) {

    if (!response.hasToolCalls()) {
        String answer = response.getResult().getOutput().getText();
        return answer;
    }

    ToolExecutionResult result =
        toolCallingManager.executeToolCalls(prompt, response);

    if (result.returnDirect()) {
        return result.conversationHistory()
            .get(result.conversationHistory().size() - 1)
            .getText();
    }

    prompt = new Prompt(result.conversationHistory(), options);

    if (iteration == MAX_ITERATIONS) {
        return "Could not complete within the allowed tool-call limit.";
```

Line-by-line concept comparison

The execution engine is the same; the prompt and boundaries shape the behaviour.

Element	First version — agentic investigation	Second version — general agent
SystemMessage	Defines investigation goal and detailed conditions	Defines general support role and tool-use guidance
UserMessage	Labels the input as CUSTOMER GOAL	Passes the question directly
MAX limit	8 cycles	5 cycles
LLM decision	Chooses next useful investigation action	Chooses whether tools are needed
Tool execution	ToolCallingManager.executeToolCalls(...)	Same
Observation	Updated conversationHistory is returned to LLM	Same
Stop	No tool calls, direct return, failure/limit	Same

Both are bounded manual loops; neither is automatically a separate Spring AI Agent object.

Worked example — how the first version behaves

Customer asks: “Check ticket T1001 and tell me whether it is within the SLA.”

Initial LLM decision

Ticket details are needed → request ticket lookup.

Ticket details are needed → request ticket lookup.

Tool result

Ticket lookup returns ticket status, priority and

Ticket lookup returns ticket status, priority and customer/account identifier.

Reassessment

SLA question remains unanswered → request

SLA question remains unanswered → request account/plan lookup.

Second tool result

Account lookup returns plan and SLA target.

Account lookup returns plan and SLA target.

Final LLM response

Compare retrieved facts and explain whether the ticket is within the SLA.

Compare retrieved facts and explain whether the ticket is within the SLA.

The second version can also perform multiple calls, but its system message does not explicitly prescribe this investigation sequence.

Important engineering notes for both implementations

What the code guarantees — and what it does not guarantee.

- The SystemMessage expresses the desired goal and policy; it is guidance to the model, not a hard enforcement mechanism.
- The LLM decides which tool(s) to request. Exposing two tools does not guarantee that both will be called.
- Independent tool calls may be requested in one response; dependent calls require the result of an earlier action.
- MAX_STEPS / MAX_ITERATIONS count loop cycles, not necessarily individual tool invocations.
- Java should enforce authorization, refund eligibility, approval, idempotency and other critical business rules.
- The prompt history preserves user input, assistant tool-call requests and tool results for the next LLM decision.
- A tool failure needs explicit handling; do not let the model invent missing facts.